



RAJALAKSHMI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ACADEMIC YEAR 2025 - 2026

SEMESTER III

ARTIFICIAL INTELLIGENCE LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240070060
NAME	DHARANI VENDAN S
PROJECT TITLE	AI POWERED SUDOKU SOLVER
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Mrs. M. Divya

Signature of Faculty In-charge

INTRODUCTION

Artificial Intelligence (AI) focuses on creating systems capable of performing tasks that require human intelligence such as reasoning, learning, problem-solving, and decision-making. One such application is solving logical puzzles like Sudoku using AI algorithms. This project aims to develop an AI-powered Sudoku Solver that utilizes backtracking algorithm to automatically generate and solve Sudoku puzzles efficiently through a user-friendly interface built with Tkinter.

- Brief overview of Artificial Intelligence concepts.
- Clearly introduce the topic, provide background context, explain why the problem matters, and highlight what the project aims to accomplish.

PROBLEM STATEMENT

The goal of the project is to design and implement an intelligent Sudoku solver that can automatically generate Sudoku puzzles of varying difficulty levels and solve them using AI-based backtracking algorithm. It should also allow users to request hints, validate solutions, and visualize the solving process.

GOAL

The expected outcome is a fully functional Sudoku-solving application with features such as puzzle generation, AI-based solving, hint generation, and difficulty level customization. The project demonstrates how artificial intelligence can be applied to constraint satisfaction problems.

- Expected result and the possibilities for the given problem statement

AI Powered Sudoku Solver

8							
	7			5			1
		2				1	
9			1		2		
	5		7	8			6
3							
	2	7				9	
				4			5

Select Difficulty:

Easy



Generate Puzzle

Solve

Hint

Clear

THEORETICAL BACKGROUND

Sudoku is a constraint satisfaction problem (CSP) where each cell must satisfy constraints: no repeated numbers in any row, column, or 3x3 subgrid. AI techniques such as backtracking search are commonly used to solve CSPs. Backtracking systematically explores possible solutions and prunes invalid configurations early, making it effective for problems like Sudoku. Other possible approaches include genetic algorithms, constraint propagation, and neural networks, but backtracking provides simplicity and accuracy.

- Theoretical Background about the problem statement as well as the algorithm.
- Literature survey of the given problem and other possible algorithms
- Justification for choosing the algorithm

ALGORITHM EXPLANATION WITH EXAMPLE

The algorithm follows a recursive backtracking approach:

1. Find the first empty cell in the Sudoku grid.
2. Try inserting numbers 1 through 9 sequentially.
3. For each number, check if it is valid within the row, column, and 3x3 box.
4. If valid, place the number and recursively attempt to fill the rest of the grid.
5. If no valid number fits, backtrack by removing the last placed number and try the next.
6. Continue until the board is completely filled.

Example:

Consider a partially filled Sudoku grid.
Suppose the first empty cell is at position **(0,0)**.

Possible numbers for (0,0) are [1, 2, 3].

- The algorithm first tries **1**.
- It continues solving other cells recursively.
- If it later encounters a conflict (no valid number for a cell), it **backtracks**: removes **1** and tries **2** next.
- This continues until the grid is completely and correctly filled.

IMPLEMENTATION AND CODE

The Sudoku Solver is implemented using Python and Tkinter for GUI. The AI logic uses the backtracking algorithm. Users can select difficulty levels (Easy, Medium, Hard, Expert), generate puzzles, request hints, and automatically solve puzzles.

Python code:

```
import tkinter as tk
from tkinter import messagebox, ttk
import random
import copy

# -----
# AI BACKTRACKING SOLVER LOGIC
```

```
# -----  
def find_empty(board):  
    for i in range(9):  
        for j in range(9):  
            if board[i][j] == 0:  
                return i, j  
    return None  
  
def valid(board, num, pos):  
    row, col = pos  
  
    # Check row  
    if num in board[row]:  
        return False  
  
    # Check column  
    for i in range(9):  
        if board[i][col] == num:  
            return False  
  
    # Check 3x3 box  
    box_x = col // 3  
    box_y = row // 3  
    for i in range(box_y * 3, box_y * 3 + 3):  
        for j in range(box_x * 3, box_x * 3 + 3):  
            if board[i][j] == num:  
                return False  
    return True  
  
def solve(board):  
    find = find_empty(board)  
    if not find:  
        return True  
    else:  
        row, col = find  
  
    for i in range(1, 10):  
        if valid(board, i, (row, col)):  
            board[row][col] = i  
            if solve(board):  
                return True  
            board[row][col] = 0  
    return False  
  
# -----
```

```

# PUZZLE GENERATOR (with difficulty)
# -----
def generate_full_board():
    """Generate a full valid Sudoku board using backtracking."""
    board = [[0 for _ in range(9)] for _ in range(9)]
    solve_random(board)
    return board

def solve_random(board):
    """Helper to generate full valid random Sudoku."""
    find = find_empty(board)
    if not find:
        return True
    else:
        row, col = find
        nums = list(range(1, 10))
        random.shuffle(nums)
        for num in nums:
            if valid(board, num, (row, col)):
                board[row][col] = num
                if solve_random(board):
                    return True
                board[row][col] = 0
        return False

def remove_cells(board, level):
    """Remove cells based on difficulty level."""
    levels = {
        "Easy": 35,
        "Medium": 45,
        "Hard": 55,
        "Expert": 60
    }
    cells_to_remove = levels.get(level, 35)
    new_board = copy.deepcopy(board)
    count = 0
    while count < cells_to_remove:
        row = random.randint(0, 8)
        col = random.randint(0, 8)
        if new_board[row][col] != 0:
            new_board[row][col] = 0
            count += 1
    return new_board

# -----

```

```

# GUI PART
# -----
class SudokuGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("AI Powered Sudoku Solver")
        self.root.configure(bg="#cfe2f3") # Pleasant blue background

        self.entries = [[None for _ in range(9)] for _ in range(9)]
        self.solution = None

        # Difficulty Selection
        self.level_var = tk.StringVar(value="Easy")

        self.create_dropdown()
        self.create_grid()
        self.create_buttons()

        self.board = [[0]*9 for _ in range(9)]

    def create_dropdown(self):
        frame = tk.Frame(self.root, bg="#cfe2f3")
        frame.pack(pady=5)
        tk.Label(frame, text="Select Difficulty:", font=("Arial", 12, "bold"),
            bg="#cfe2f3").pack(side=tk.LEFT)
        combo = ttk.Combobox(frame, textvariable=self.level_var, values=["Easy", "Medium", "Hard",
            "Expert"], state="readonly")
        combo.pack(side=tk.LEFT, padx=5)

    def create_grid(self):
        frame = tk.Frame(self.root, bg="#cfe2f3")
        frame.pack()

        for i in range(9):
            for j in range(9):
                e = tk.Entry(frame, width=3, font=("Arial", 18), justify="center",
                    borderwidth=2, relief="ridge", fg="#004080", bg="#e6f0ff")
                e.grid(row=i, column=j, padx=1, pady=1)
                self.entries[i][j] = e

        # Thicker borders for boxes
        if i % 3 == 0 and i != 0:
            e.grid(pady=(4, 1))
        if j % 3 == 0 and j != 0:
            e.grid(padx=(4, 1))

```

```

def create_buttons(self):
    frame = tk.Frame(self.root, bg="#cfe2f3")
    frame.pack(pady=10)

    tk.Button(frame, text="Generate Puzzle", command=self.generate_puzzle, bg="#007acc",
              fg="white", font=("Arial", 10, "bold")).grid(row=0, column=0, padx=5)
    tk.Button(frame, text="Solve", command=self.solve_puzzle, bg="#007acc", fg="white",
              font=("Arial", 10, "bold")).grid(row=0, column=1, padx=5)
    tk.Button(frame, text="Hint", command=self.show_hint, bg="#007acc", fg="white",
              font=("Arial", 10, "bold")).grid(row=0, column=2, padx=5)
    tk.Button(frame, text="Clear", command=self.clear, bg="#007acc", fg="white", font=("Arial",
    10, "bold")).grid(row=0, column=3, padx=5)

def set_board(self, board):
    for i in range(9):
        for j in range(9):
            self.entries[i][j].delete(0, tk.END)
            if board[i][j] != 0:
                self.entries[i][j].insert(0, str(board[i][j]))
                self.entries[i][j].config(fg="#000066") # dark blue numbers
            else:
                self.entries[i][j].config(fg="#004080")

def get_board(self):
    board = []
    for i in range(9):
        row = []
        for j in range(9):
            val = self.entries[i][j].get()
            row.append(int(val) if val else 0)
        board.append(row)
    return board

def generate_puzzle(self):
    level = self.level_var.get()
    full_board = generate_full_board()
    puzzle = remove_cells(full_board, level)
    self.solution = copy.deepcopy(full_board)
    self.board = puzzle
    self.set_board(puzzle)
    messagebox.showinfo("Sudoku Generated", f'{level} level puzzle generated!')

def solve_puzzle(self):
    board = self.get_board()
    if solve(board):
        self.set_board(board)

```



```

messagebox.showinfo("AI Solver", "Sudoku solved successfully!")
else:
messagebox.showwarning("AI Solver", "No solution exists!")

def show_hint(self):
    """Show one correct value for an empty cell."""
    if not self.solution:
        messagebox.showwarning("Hint", "Please generate a puzzle first!")
    return

    board = self.get_board()
    for i in range(9):
        for j in range(9):
            if board[i][j] == 0:
                correct_value = self.solution[i][j]
                self.entries[i][j].delete(0, tk.END)
                self.entries[i][j].insert(0, str(correct_value))
                self.entries[i][j].config(fg="#0066cc")
                messagebox.showinfo("Hint", f"Hint placed at row {i+1}, column {j+1}.")
            return
        messagebox.showinfo("Hint", "No empty cells left!")

def clear(self):
    for i in range(9):
        for j in range(9):
            self.entries[i][j].delete(0, tk.END)
    self.solution = None

# -----
# MAIN FUNCTION
# -----
if __name__ == "__main__":
    root = tk.Tk()
    SudokuGUI(root)
    root.mainloop()

```

OUTPUT:

- ❖ When you run the python code , displays the output screen .



- ❖ Select how the level is difficultly , either easy or medium or hard or expert level.
- ❖ When you turn on the AI mode ,when you click on AI step it will automatically

generate the number to be filled in the next empty box by backtracking algorithm.

- ❖ There is also a manual method ,you can try, if you place the wrong number in the particular place ,it will show in red color.
- ❖ When you click the Solve the puzzle is solved.
- ❖ When the clear button is clicked it cleared numbers you or AI filled in the puzzle and to start from first.

The output includes a Tkinter-based interactive Sudoku board where users can:

- Generate Sudoku puzzles with different difficulty levels.
- Request hints for specific cells.
- Automatically solve puzzles using AI.
- View instant validation of results.

RESULTS AND FUTURE ENHANCEMENT

The project successfully demonstrates AI-based problem solving in a real-time interactive application. The backtracking algorithm performs accurately and efficiently for 9x9 Sudoku boards. Future improvements could include:

- Step-by-step solving visualization.
- Integration of image recognition for handwritten Sudoku.
- Deployment as a web or mobile app.
- Use of machine learning to optimize puzzle generation.

Git hub link of the project and report:
https://github.com/dharanivendan-star/AI-DV-MINI-PROJECT

REFERENCES

1. Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach.
2. Python.org - Official Python Documentation.
3. TkDocs - Tkinter GUI Programming Guide.
4. GeeksforGeeks - Backtracking Algorithm in Python.
5. Towards Data Science - Solving Sudoku using AI Techniques.

- At least 5 Resources that you referred (May be books/on-line resources (E-Books, standard Videos and blogs Strictly no conversational agents like ChatGPT, Gemini etc.,)